MODULE 16 · DOMAIN 2 - CONTAINER ORCHESTRATION

Service mesh

Proxies that give every call encryption, identity, retries and metrics, without touching the code.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 16 of 35

admin@sthithapragna.com

A service mesh moves **networking concerns out of the application** and into a layer of proxies.

What a mesh gives you

AREA	EXAMPLES
Traffic management	Retries, timeouts, circuit breaking, traffic splitting, mirroring
Security	Mutual TLS on every call, workload identity, authorisation policies
Observability	Request rate, errors and latency for every service-to-service call

ON THE EXAM

Same features in every language, with no code changes. That uniformity is the selling point.

Data plane versus control plane

DATA PLANE

Handles every request

- Proxies next to or near each workload
- Encrypt, route, retry, measure
- Examples: Envoy, ztunnel, linkerd2-proxy
- Keeps working with its last config

CONTROL PLANE

Configures the proxies

- Pushes routing and policy to proxies
- Issues and rotates certificates
- Examples: istiod, Linkerd's control plane
- Handles no application traffic

THE TIEBREAKER

Same split as Kubernetes itself: the control plane decides, the data plane does the work.

Sidecar versus sidecarless

SIDECAR

A proxy in every Pod

- Full layer 7 features per Pod
- CPU and memory cost in every Pod
- Upgrading means restarting Pods
- The classic design

SIDECARLESS

Proxies per node

- A shared proxy per node for L4 and mTLS
- Layer 7 proxies only where needed
- Far less overhead, no Pod restarts
- Istio ambient mode, Cilium

THE TIEBREAKER

Mesh overhead too high, or Pod restarts on every upgrade: consider a sidecarless design.

Istio ambient mode

- **ztunnel:** a per-node proxy giving every Pod mTLS, identity and layer 4 policy.
- **Waypoint proxies:** Envoy, deployed only where layer 7 routing or policy is needed.
- **Generally available since Istio 1.24,** November 2024. Sidecar mode is still supported.
- Opt in by labelling a namespace. No change to the Pods themselves.

> ON THE EXAM
>
> Older material presents sidecars as the only way to run a mesh. That is no longer true.

Meshes and proxies you should recognise

NAME	KNOWN FOR
Envoy	The layer 7 proxy inside many meshes and gateways. CNCF graduated
Istio	Feature-rich mesh on Envoy; sidecar or ambient. CNCF graduated
Linkerd	Simplicity; its own lightweight proxy written in Rust. CNCF graduated
Cilium	eBPF-based, sidecarless mesh from the network plugin
Kuma, Consul	Envoy-based meshes from Kong and HashiCorp

ON THE EXAM

Envoy is a proxy, not a mesh. Istio and Linkerd are meshes. Questions like to blur that line.

Mutual TLS: **both sides prove who they are**, and everything between them is encrypted.

admin@sthithapragna.com

TLS versus mutual TLS

TLS

The server proves itself

- Client checks the server's certificate
- Server does not know who the client is
- Encrypted in transit
- Typical HTTPS on the web

MUTUAL TLS

Both sides prove themselves

- Both present certificates
- Each side knows the other's identity
- Encrypted in transit
- Service-to-service in a mesh

THE TIEBREAKER

'Encrypt and authenticate both sides of every internal call': mutual TLS, usually via a mesh.

Workload identity

- **Each workload gets a certificate** tied to its ServiceAccount.
- **SPIFFE format:** spiffe://cluster.local/ns/quayside/sa/booking-api.
- **Short-lived and rotated automatically** by the control plane. Nobody handles keys.
- **Policies use identity, not IP addresses,** which change constantly in Kubernetes.

> ON THE EXAM
>
> SPIFFE and its runtime, SPIRE, are graduated CNCF projects for workload identity.

One call through the mesh

01

App sends

The payment worker sends plain HTTP to booking-api.

02

Proxy intercepts

Its local proxy picks up the call.

03

Handshake

Proxies exchange mesh-issued certificates.

04

Policy

Identity checked: may payments call bookings?

05

Deliver

Encrypted across the network; plain HTTP to the app.

ON THE EXAM

Neither application knows TLS happened. That is the whole point.

What a mesh costs

- **Latency:** every call passes through one or two extra proxies.
- **Resources:** CPU and memory for proxies, most of all with sidecars.
- **Complexity:** another control plane to run, upgrade and debug.
- **Worth it** with many services, strict encryption or zero-trust needs, or poor visibility.

> ON THE EXAM
>
> A mesh is not a default. A handful of services may be better served by libraries and NetworkPolicy.

Currency check: from SMI to GAMMA

- **SMI, the Service Mesh Interface,** was archived by the CNCF in October 2023.
- **GAMMA** uses the Gateway API for mesh traffic, inside the cluster.
- **In the Gateway API's standard channel since v1.1**, released in 2024.
- Older material calls SMI 'the' mesh standard. Its maintainers moved to GAMMA.

> ON THE EXAM
>
> Portable mesh traffic rules today: Gateway API HTTPRoutes, attached to a Service.

GAMMA: an HTTPRoute attached to a Service

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: {name: booking-api-split, namespace: quayside}
spec:
  parentRefs:
  - {group: "", kind: Service, name: booking-api, port: 80}
  rules:
  - backendRefs:
    - {name: booking-api-v2, port: 80, weight: 90}
    - {name: booking-api-v3, port: 80, weight: 10}
```

6 The parent is a Service, not a Gateway: this is mesh traffic.

9-10 Calls to booking-api are split 90 to 10 between versions.

THE TAKEAWAY

Same HTTPRoute as module 15. Parent a Gateway for traffic in; parent a Service for traffic between services.

> Quayside's security team requires every call between its 40 internal services to be encrypted, with both caller and receiver authenticated.
>
> The services are written in five languages, and teams cannot change their code this year.

Which approach meets the requirement?

A NetworkPolicies allowing only the expected callers

B TLS termination at the Ingress or Gateway

C Certificates in Secrets, loaded by each service

D A service mesh enforcing mutual TLS

◆ PAUSE AND DECIDE

Which approach meets the requirement?

A NetworkPolicies allowing only the expected callers

Restricts who connects, but encrypts nothing and proves no identity.

B TLS termination at the Ingress or Gateway

Covers traffic coming in, not calls between services.

C Certificates in Secrets, loaded by each service

Needs code changes in five languages, which is ruled out.

D A service mesh enforcing mutual TLS

Encrypts and authenticates both sides, with no code changes.

THE SIGNAL

Encrypt and authenticate every internal call, without code changes: mesh mTLS.

> Quayside runs a sidecar mesh across 400 small Pods. The proxies use more memory than many of the apps, and every mesh upgrade restarts every Pod.
>
> The team still needs mutual TLS everywhere, and layer 7 routing for only a few services.

What change best addresses this?

A Add memory to every node and keep the sidecars

B Turn off mutual TLS to reduce proxy load

C Move to a sidecarless mode such as Istio ambient

D Run Envoy as a DaemonSet, with no mesh control plane

◆ PAUSE AND DECIDE

What change best addresses this?

A Add memory to every node and keep the sidecars

Pays for the problem instead of removing it; restarts remain.

B Turn off mutual TLS to reduce proxy load

Drops the one requirement that must stay.

C Move to a sidecarless mode such as Istio ambient

Per-node mTLS, waypoints only where needed, no Pod restarts.

D Run Envoy as a DaemonSet, with no mesh control plane

No certificates, identity or configuration: not a mesh.

THE SIGNAL

Sidecar overhead and restarts, mTLS everywhere, L7 in a few places: sidecarless.

> An internal runbook from 2021 says to use an SMI TrafficSplit object for canary releases inside Quayside's mesh.
>
> In 2026 the team wants a supported, portable way to split internal traffic 90 to 10 between versions.

What should they use now?

A. A Gateway API HTTPRoute whose parent is the Service

B. SMI TrafficSplit, which is still the mesh standard

C. Ingress annotations from the ingress controller

D. The Deployment's maxSurge and maxUnavailable

◆ PAUSE AND DECIDE

What should they use now?

A A Gateway API HTTPRoute whose parent is the Service

GAMMA: the standard, portable way, since Gateway API v1.1.

B SMI TrafficSplit, which is still the mesh standard

SMI was archived in 2023.

C Ingress annotations from the ingress controller

Ingress handles traffic coming in, not calls between services.

D The Deployment's maxSurge and maxUnavailable

Paces a rollout; gives no steady 90 to 10 split.

THE SIGNAL

Mesh traffic splitting today: GAMMA, meaning an HTTPRoute with a Service parent.

> istiod, Istio's control plane, crashes and stays down for fifteen minutes. The data plane proxies are all healthy.
>
> Services were exchanging mutually authenticated traffic when it failed.

What happens to that service-to-service traffic?

A All traffic between services stops at once

B It keeps flowing on the proxies' last config

C It falls back to plain, unencrypted HTTP traffic

D Pods restart without their mesh proxies

◆ PAUSE AND DECIDE

What happens to that service-to-service traffic?

A. All traffic between services stops at once
The control plane carries no traffic, so nothing stops.

B. It keeps flowing on the proxies' last config
The data plane runs on; only changes and new certificates wait.

C. It falls back to plain, unencrypted HTTP traffic
Proxies keep their existing certificates; no silent downgrade.

D. Pods restart without their mesh proxies
Nothing restarts Pods because the control plane is down.

THE SIGNAL

Control plane down: change stops, traffic continues. The same rule as Kubernetes.

Module 16 in five lines

- A mesh moves **encryption, identity, retries and metrics** into proxies, with no code changes.
- **Data plane** proxies carry traffic; the **control plane** configures them and issues certificates.
- **Sidecar**: a proxy per Pod. **Sidecarless**: per-node proxies, e.g. Istio ambient, GA in 1.24.
- **Mutual TLS**: both sides authenticated and encrypted, identity tied to ServiceAccounts.
- **SMI is archived;** mesh routing now uses the Gateway API through **GAMMA**.

Mesh identity is built on ServiceAccounts. Next: **access control**, who may do what in a cluster.